UNIVERSITAS PAHLAWAN TUANKU TAMBUSAI
FAKULTAS TEKNIK — PROGRAM STUDI TEKNIK INFORMATIKA

LEMBAR KERJA MAHASISWA

Pertemuan 6 · Pewarisan dan Method Khusus

Nama	Aidiel Putra Perdana	**NIM**	2555201006
Kelas	A	**Tanggal**	11 November 2026
Tautan repositori	https://github.com/Aidiel1429/PBO-UP-2026.git		

A. KUIS AWAL — MENJEMBATANI DARI PERTEMUAN 5 (10 MENIT)

1. Sebutkan uji pembeda antara komposisi dan agregasi.
Uji pembedanya terletak pada daur hidup dan kepemilikan. Bila objek induk dihapus dan objek anak ikut kehilangan makna/lenyap, itu adalah komposisi (misal: Sewa dan BarisSewa). Bila objek anak tetap berdiri sendiri dan masih berguna bagi pihak lain, itu adalah agregasi (misal: BarisSewa merujuk ke AlatTani)

2. Mengapa BarisSewa menyimpan salinan tarif, padahal ia sudah menyimpan rujukan ke alatnya?
Karena tarif sewa alat dapat berubah dari waktu ke waktu (musim ke musim), tetapi angka pada nota transaksi yang sudah terlanjur dibayarkan petani di masa lalu tidak boleh ikut berubah

3. Mengapa property baris mengembalikan tuple, bukan daftar?
Agar koleksi baris sewa terbungkus aman dan tidak dapat dimodifikasi secara langsung dari luar (mencegah kebocoran komposisi seperti penambahan data tanpa melalui validasi method tambah_alat)

4. Tulis multiplisitas relasi Sewa – BarisSewa di kedua ujungnya.

1. Di ujung kelas Sewa: **1** (satu transaksi sewa).
2. Di ujung kelas BarisSewa: **1..10** (memuat satu sampai sepuluh baris sewa alat)

B. PENGAMATAN PRAKTIKUM

B.1 super() yang terlupa (Langkah 3)

Pertanyaan	Jawaban Anda
Apakah objeknya berhasil dibuat? Apakah ada galat saat itu?	Ya, objek berhasil dibuat tanpa ada galat sama sekali saat instansiasi awal dilakukan
Apa isi vars(t), dan berapa atribut yang seharusnya ada?	vars(t) berisi ['_tenaga_kuda'] (hanya ada 1 atribut). Seharusnya objek tersebut memiliki 4 atribut (termasuk atribut milik kelas induk seperti _kode, _nama, dan _tarif_harian)
Method mana yang runtuh, dan atribut apa yang dicarinya?	Method yang runtuh adalah keterangan() atau biaya_sewa(), yang mencari atribut _kode dan _tarif_harian milik kelas induk yang gagal diinisialisasi
Dengan kalimat sendiri: apa sesungguhnya yang dikerjakan super().__init__()?	super().__init__() bertugas untuk mengeksekusi konstruktor kelas induk guna membangun dan menginisialisasi seluruh atribut dasar serta aturan validasi induk ke dalam objek turunan

B.2 __eq__ tanpa __hash__ (Langkah 7)

Pertanyaan	Jawaban Anda
Berapa nilai AlatTani.__hash__yang ditampilkan?	None
Tiga tindakan apa yang gagal, dan apa pesan galatnya?	1. Dimasukkan ke set → Gagal, pesan galat: unhashable type: 'AlatTani'. 2. Dijadikan kunci dict → Gagal, pesan galat: unhashable type: 'AlatTani'. 3. Dicari dengan in → Gagal, pesan galat: unhashable type: 'AlatTani'.
Mengapa Python mematikan __hash__ begitu __eq__ditulis ulang?	Karena Python menganggap aturan kesamaan objek sudah berubah, sehingga aturan hash yang lama tidak lagi valid dan harus didefinisikan ulang secara manual
Aturan apa yang harus dipenuhi __eq__ dan __hash__agar konsisten?	Dua objek yang dinyatakan sama oleh __eq__ wajib menghasilkan nilai hash yang sama persis

B.3 Dua cara perkakas menangkap kesalahan

Berkas	Kode Aturan Ruff	Menunjuk Gejala atau Penyakit?
`lupa_super.py`	E302 / F821 / *atau peringatan inisialisasi* (menyesuaikan aturan mypy/ruff terkait super())	Menunjukkan gejala bahwa kelas turunan gagal memanggil super().__init__(), sehingga atribut dari kelas induk tidak terinisialisasi dan gagal dibentuk
`eq_tanpa_hash.py`	B023 / *aturan hash/eq* (__eq__ tanpa __hash__)	Menunjukkan gejala bahwa kelas mendefinisikan method perbandingan kesamaan (__eq__) tetapi tidak mendefinisikan __hash__, yang membuat objek tidak bisa dimasukkan ke dalam struktur data berbasis set atau dijadikan kunci dictionary

1. Ruff menangkap keduanya, tetapi dengan cara yang berbeda. Jelaskan perbedaannya, dan apa artinya bagi cara Anda membaca peringatan perkakas.

Ruff mendeteksi lupa_super.py sebagai kesalahan akses atribut karena induk tidak diinisialisasi, sedangkan eq_tanpa_hash.py ditandai sebagai pelanggaran aturan integritas objek hashing; artinya, perkakas berfungsi sebagai deteksi dini untuk mencegah *bug* logika dan *crash* saat program berjalan

C. AKTIVITAS KELOMPOK: KAPAN PEWARISAN MERUGIKAN

Anggota kelompok: Aidiel Putra Perdana, Budi Mukhoyyar, Fitri Nabila_____

C.1 Uji "adalah sejenis" pada proyek kelompok

Pasangan Kelas	Kalimat "A adalah sejenis B"	Masuk Akal? Relasi yang Tepat?
Traktor & AlatTani	"Traktor adalah sejenis AlatTani"	Masuk akal. Relasi yang tepat adalah Pewarisan (Inheritance) karena Traktor adalah spesialisasi dari alat tani
PompaAir & Traktor	"PompaAir adalah sejenis Traktor"	Tidak masuk akal. Relasi yang tepat bukan pewarisan, melainkan Asosiasi / Kelas Sejajar (keduanya sama-sama turunan langsung dari AlatTani)
Mesin / Motor Penggerak & PompaAir	"PompaAir adalah sejenis Mesin Penggerak"	Tidak masuk akal. Relasi yang tepat adalah Komposisi (Composition / "punya"), karena PompaAir *memiliki* komponen mesin/debit air di dalamnya, bukan mewarisi sifat mesin secara umum

C.2 Kemiripan yang sebaiknya diselesaikan dengan komposisi

Sebutkan satu, beserta alasannya. Butir ini bernilai paling tinggi.

1. Contoh Kasus pada Proyek: Atribut pengelolaan spesifikasi mesin atau komponen fungsional tambahan (seperti kapasitas bahan bakar / konsumsi solar pada PompaAir atau operator pada Traktor).
2. Alasan: Fitur tersebut lebih tepat diselesaikan dengan komposisi (menyertakan objek/atribut pendukung di dalam kelas) alih-alih dipaksakan menjadi hierarki pewarisan yang dalam. Hal ini mencegah terjadinya *inheritance abuse* (pewarisan dipaksakan), membuat kelas lebih fleksibel, serta menjaga agar struktur kode tidak kaku atau terlalu rumit saat terjadi perubahan spesifikasi komponen.

D. REFLEKSI PENUTUP

1. Validasi tarif yang Anda tulis di Pertemuan 3 kini menjaga tiga kelas baru tanpa satu baris pun disalin. Menurut Anda apa risikonya bila suatu hari validasi itu perlu berbeda untuk salah satu turunan?

Risikonya adalah kita terpaksa merusak prinsip pewarisan dengan melakukan *override* pada setter kelas induk atau menulis ulang logika validasi khusus di kelas turunan tersebut, yang dapat memicu duplikasi kode (*code duplication*) dan membuat hierarki kelas menjadi kaku serta rentan terhadap *bug* saat terjadi perubahan aturan bisnis di masa depan

2. Sebutkan satu hal yang belum Anda pahami, dan pertanyaan untuk Pertemuan 7.

1. **Hal yang belum dipahami: Penerapan polimorfisme tingkat lanjut pada objek antarmuka yang melibatkan banyak kelas turunan majemuk.**
2. **Pertanyaan untuk Pertemuan 7: Bagaimana cara terbaik merancang kelas abstrak (*abstract base class*) di Python agar aturan implementasi di kelas turunan bisa dipaksa secara otomatis sejak awal perancangan?**

Paraf Asisten / Dosen	Tanda Tangan Mahasiswa
.....	.....